Ex.No.2	GENERATIVE ADVERSARIAL NETWORK FOR CODE GENERATION	URK23CS8018
27-08-2026		

Aim:

To develop a simplified GAN-style system for generating Python code using CodeLlama and classifying real and generated code using a machine learning classifier.

Procedure:

Step 1: Create a dataset of real Python code snippets.
Step 2: Use CodeLlama in Ollama to generate Python code.
Step 3: Combine real and generated code samples.
Step 4: Convert the code into numerical features using TF-IDF.
Step 5: Train a Logistic Regression classifier as the discriminator.
Step 6: Classify the real and generated code.
Step 7: Display the prediction and confidence values.□

Question:

Generative Adversarial Network for Code Generation

Construct a simplified GAN-style architecture for code generation where an LLM acts as the generator and a machine-learning classifier acts as the discriminator.

Program:

```
import ollama
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
real_code_samples = [
"""def bubble_sort(arr):
    n = len(arr)

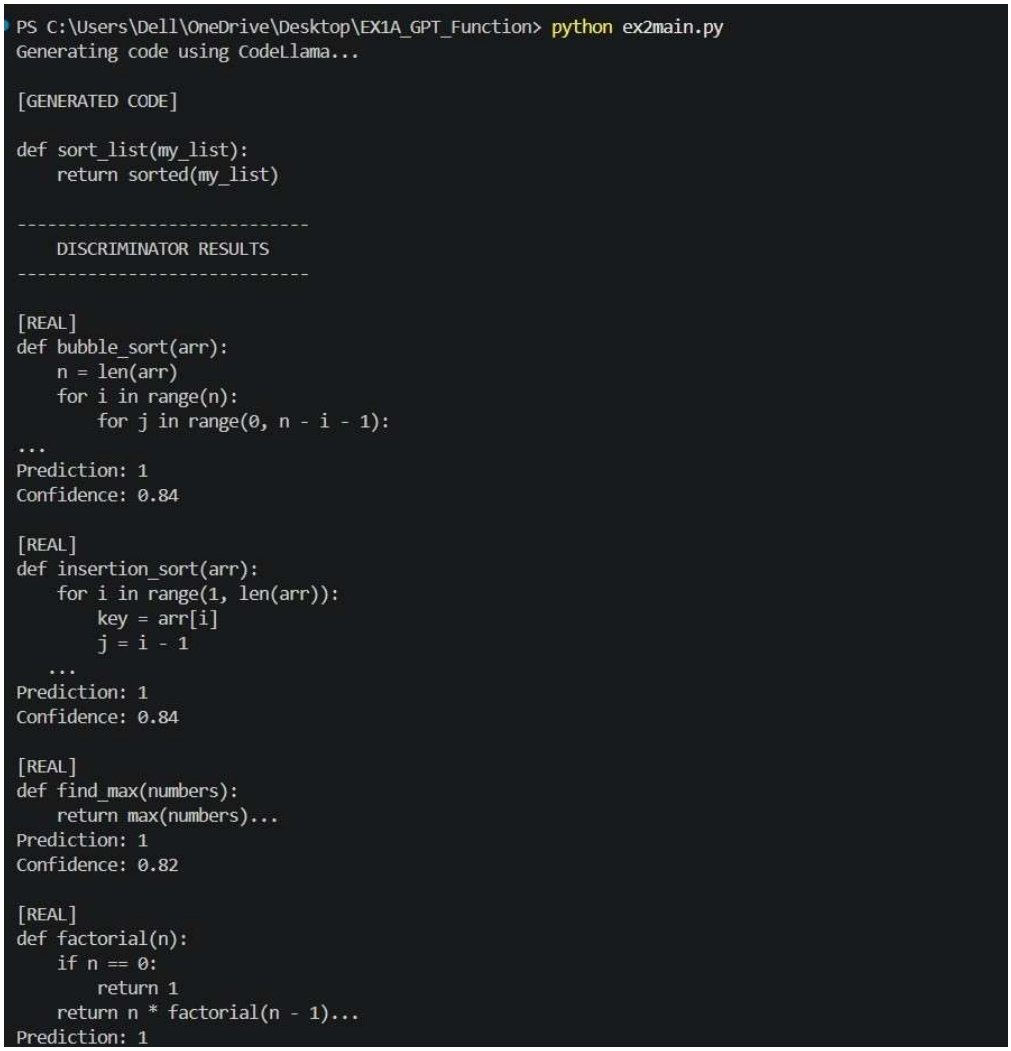
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr
""",
"""def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

```
    """,
    """def find_max(numbers):
        return max(numbers)
    """,
    """def factorial(n):
        if n == 0:
            return 1
        return n * factorial(n - 1)
    """
    ]
def generate_code():
    prompt = """
    Generate only valid Python code.
    Write a Python function that sorts a list of numbers.
    Do not provide explanations.
    Return only Python code.
    """
    response = ollama.chat(
        model="codellama:7b",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
    )
    return response["message"]["content"]
print("Generating code using CodeLlama...\n")
generated_code = generate_code()
print("[GENERATED CODE]\n")
print(generated_code)
texts = real_code_samples + [generated_code]
labels = [1] * len(real_code_samples) + [0]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)
discriminator = LogisticRegression()
discriminator.fit(X, labels)
print("\n==============================")
print("DISCRIMINATOR RESULTS")
print("==============================\n")
for code in real_code_samples:
    code_vector = vectorizer.transform([code])
    prediction = discriminator.predict(code_vector)[0]
    confidence = discriminator.predict_proba(
        code_vector
    )[0][1]
```

```
print("[REAL]") print(code[:100] + "...") print("Prediction:", prediction)
    print("Confidence:", round(confidence, 2))
    print()
generated_vector = vectorizer.transform(
    [generated_code]
)
prediction = discriminator.predict(
    generated_vector
)[0]
confidence = discriminator.predict_proba(
    generated_vector
)[0][0]
print("[GENERATED]")
print(generated_code[:150] + "...") print("Prediction:", prediction) print("Confidence:", round(confidence, 2))
```

Output Screenshot:



```
Prediction: 1
Confidence: 0.84

[REAL]
def find_max(numbers):
    return max(numbers)...
Prediction: 1
Confidence: 0.82

[REAL]
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)...
Prediction: 1
Confidence: 0.82

[GENERATED]
def sort_list(my_list):
    return sorted(my_list)...
Prediction: 1
Confidence: 0.32
```

Link:

https://drive.google.com/file/d/1rUlnR-pcqdWA5-u9trOT_J_3jbhQxPJV/view?usp=sharing

Result:

Thus, a simplified GAN-style architecture was successfully implemented to generate Python code and classify real and generated code samples.